Name ..... Roll# .....

Scenario: Operation Reconciliation Accord — Diplomatic Treaty Negotiation System

After years of diplomatic tension, two sovereign factions — **Faction A** and **Faction B** — have agreed to sit at the negotiation table. They must draft a **"Reconciliation Accord"** that covers three critical domains: **Economics, Security,** and **Human Rights.** Each faction has a current state (resources) and non-negotiable **Red Lines** — minimum acceptable values per domain. You are hired to design a C++ system modelling these factions, their treaty clauses, and the negotiation process. (min is minimum)

Q1 — Reconciliation Accord Negotiation System (use exactly these in main())

Attribute	Faction A (Aurantia)	Faction B (Borean)
economicWealth	80	60
militaryStrength	70	90
rightsIndex	50	40
Red Line: economy	min 40	min 30
Red Line: military	min 30	min 45
Red Line: rights	min 35	min 30

Q1a — The Faction Class: Encapsulation (10 Marks) [CLO 1]

What to implement:

Write a **Faction** class that stores a faction's current state and its Red Lines. All six attributes (**economicWealth, militaryStrength, rightsIndex** and the three matching red-line *minimums*) must be **private.** Provide public getters for all six and public setters only for the three state attributes. The setter for each state attribute must **reject** any new value that would drop below the faction's own Red Line for that attribute — it must print an error and leave the value unchanged.

Expected output when Q1a is complete:

```
// In main(), after construction:
factionA.setEconomicWealth(20);  // below red line of 40
>> [REJECTED] economicWealth cannot drop below Red Line (40). Value
unchanged.
factionA.setEconomicWealth(60);  // valid
>> economicWealth updated to 60.
```

Q1b — Abstract Clause Hierarchy: Inheritance & Abstraction (10 Marks) [CLO 1]

What to implement:

Write an abstract base class **TreatyClause** with a pure virtual method **evaluateCompromise(Faction& f1, Faction& f2)** that returns **bool**. Then derive the three concrete clause classes below. Each class must store the clause name and override **evaluateCompromise()**.

Clause Class	Compromise Algorithm	Accord Fails If
EconomicClause	Compute the average of both factions' economicWealth. Set both factions' economicWealth to that average (each loses or gains accordingly).	The averaged value would fall below either faction's Red Line for economy.
SecurityClause	Each faction reduces its militaryStrength by 10 units as a sign of mutual de-escalation.	After the 10-unit reduction, either faction's militaryStrength would fall below its Red Line.